



École Nationale des Sciences Appliquées Tétouan

Module : Virtualisation, Cloud et Edge Computing
Encadré par : Pr. BENALY Mohamed

Rapport Technique

Projet 09 : Déploiement et traitement distribué
avec Edge Computing

KUNAKA Daniel

MOUHAFID Hafsa

SABOR Laila

DIALLO Abdoul-Moumouni

SIMPORE Taobata

Données & Ingestion (ESP32 / Microservice A)

DevOps Edge (K3s / Raspberry Pi)

Traitement Edge (Microservice B)

Backend Cloud (API FastAPI / Azure)

Synchronisation & Dashboard (Microservice C)

Mai 2026

Table des matières

1	Introduction et contexte	3
1.1	Problématique	3
1.2	Objectifs du projet	3
1.3	Outils et technologies	3
2	Gestion de projet : Approche Scrum	3
2.1	Cadre méthodologique	3
2.2	Cérémonies Scrum	4
2.3	Estimation par Story Points	4
3	Architecture globale	5
3.1	Vue d'ensemble	5
3.2	Flux de données bout en bout	5
3.3	Namespace K3s	5
4	Simulateur IoT et Microservice A	6
4.1	Philosophie Edge Computing à la source	6
4.2	Structure des données (Data Contract)	6
4.3	Génération modélisée des données	7
4.4	Algorithmes de filtrage à la source	7
4.5	Sérialisation JSON et transmission	7
5	Infrastructure DevOps Edge (K3s)	8
5.1	Objectif	8
5.2	Installation de K3s : Étapes détaillées	8
5.3	Installation de Docker	10
5.4	Déploiement des microservices	11
5.5	Distinction Docker vs K3s	11
6	Microservice B : Traitement Edge	12
6.1	Rôle dans l'architecture	12
6.2	Détection d'alertes : <code>detecter_alertes()</code>	12
6.3	Agrégation : <code>calculer_agregat()</code>	13
6.4	Endpoint POST <code>/traiter</code>	14
6.5	Déploiement et liveness probe	14
7	API Cloud et VM Azure	15
7.1	Infrastructure Azure	15
7.2	Stack déployée	15
7.3	API FastAPI : Endpoints	16
7.4	Schéma de données PostgreSQL	16
8	Sécurisation des communications	17
8.1	Architecture de sécurité	17
8.2	Configuration Nginx	18

9	Microservice C et Dashboard EdgePulse	19
9.1	Microservice C : Synchronisation Edge-Cloud	19
9.2	Dashboard EdgePulse	19
10	Résultats et comparaison des performances	22
10.1	Latences mesurées	22
10.2	Analyse	22
10.3	Réduction du trafic réseau	22
11	Conclusion	22

1. Introduction et contexte

1.1. Problématique

Les architectures Cloud traditionnelles centralisent tous les traitements sur des serveurs distants. Dans un contexte IoT, cela implique que chaque donnée capteur effectue un aller-retour complet vers le Cloud avant qu'une décision puisse être prise : une latence typiquement supérieure à 100 ms. Pour des cas d'usage critiques (détection d'anomalies, alertes temps réel), cette latence est inacceptable.

L'Edge Computing répond à cette problématique en déportant une partie des traitements au plus proche des sources de données, réduisant la latence à moins de 10 ms pour les décisions locales tout en conservant le Cloud pour le stockage à long terme et l'analyse globale.

1.2. Objectifs du projet

Ce projet vise à implémenter une architecture hybride Edge-Cloud complète et fonctionnelle :

- Déployer des microservices sur un nœud Edge (Raspberry Pi 5)
- Assurer la communication sécurisée Edge-Cloud via une API REST HTTPS
- Réduire la latence de traitement grâce au traitement local
- Comparer les performances Edge (<10 ms) vs Cloud (>100 ms)
- Sécuriser toutes les communications (TLS, authentification)

1.3. Outils et technologies

Technologie	Usage
Raspberry Pi 5 (16 Go RAM)	Nœud Edge physique
ESP32 (microcontrôleur)	Génération des données capteurs
Docker	Conteneurisation des microservices
K3s (Kubernetes léger)	Orchestration sur le Pi
FastAPI (Python)	API REST Cloud
PostgreSQL 16	Base de données Cloud
Nginx + Let's Encrypt	Reverse proxy + certificat SSL
Microsoft Azure	Infrastructure Cloud (VM Ubuntu)
Streamlit + Plotly	Dashboard de visualisation

2. Gestion de projet : Approche Scrum

2.1. Cadre méthodologique

Face à la complexité de l'intégration de composants matériels (Edge) et logiciels (Cloud), nous avons opté pour une planification méthodologique en implémentant le framework Agile Scrum. Un **sprint intensif d'une semaine** (du 5 mai au 10 mai 2026) a été adopté : ce *timeboxing* force une livraison incrémentale et identifie immédiatement les goulots d'étranglement entre les composants.

2.2. Cérémonies Scrum

- **Sprint Planning (5 mai)** : définition des objectifs et formalisation de la *Definition of Done* pour chaque composant. Exemple : “une tâche d’ingestion est terminée si la donnée est reçue, formatée en JSON, et logguée sans erreur de typage”.
- **Mêlées quotidiennes** : points de synchronisation pour gérer les dépendances séquentielles (logique *Finish-to-Start*). Le pipeline d’ingestion ne pouvait pas être validé tant que le contrat de données n’était pas figé.
- **Sprint Review et Rétrospective (10 mai)** : validation du flux de données complet de bout en bout, avec démonstration d’un pipeline stable et sans latence critique.

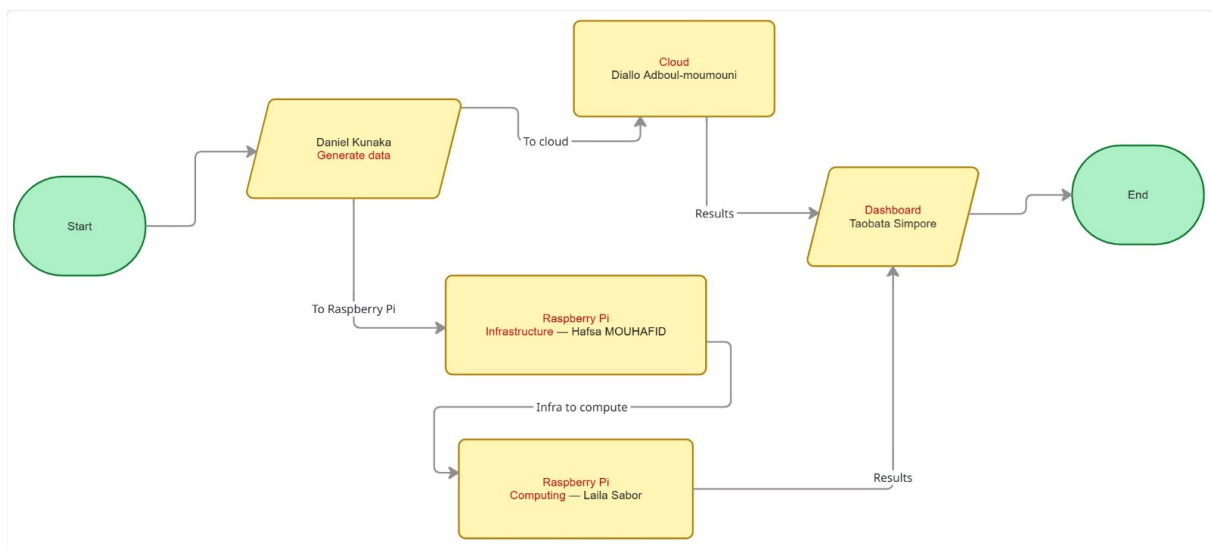


FIGURE 1 – Vue de la planification

2.3. Estimation par Story Points

L’effort technique a été quantifié via la **suite de Fibonacci** (1, 2, 3, 5, 8, 13) :

Type de tâche	Story Points	Justification
Tâches d’ingestion	5 pts	Complexité moyenne, algorithmes C++
Planification / cérémonies	0-1 pt	Marqueurs administratifs
Infrastructure K3s	8 pts	Configuration spécifique ARM64
API Cloud + déploiement	8 pts	Stack complète FastAPI + Azure
Dashboard Streamlit	5 pts	Visualisation multi-sources

Lien vers le travail de planification : miro.com

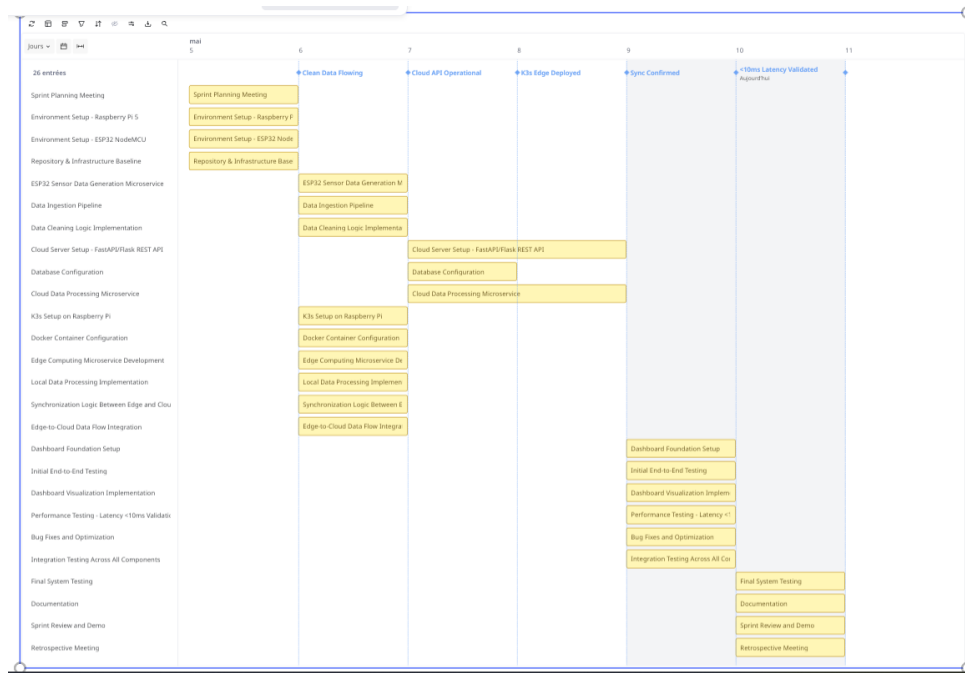


FIGURE 2 – Exécution du projet suivant la planification

3. Architecture globale

3.1. Vue d'ensemble

L'architecture repose sur deux machines physiques distinctes interconnectées par Internet :

Machine	Rôle	Accès	Adresse
Raspberry Pi 5	Nœud Edge	Raspberry Pi Connect	réseau local
VM Azure	Serveur Cloud	SSH + HTTPS	20.164.76.70

3.2. Flux de données bout en bout

1. L'**ESP32** génère des données capteurs chaque seconde et les envoie via HTTP POST au Microservice A sur le Raspberry Pi.
2. Le **Microservice A** reçoit et bufferise les données brutes.
3. Le **Microservice B** filtre en temps réel (<10 ms), génère des alertes locales et agrège les données par moyenne sur 5 minutes.
4. Le **Microservice C** envoie les agrégats au Cloud Azure toutes les 5 minutes via HTTPS avec backoff exponentiel en cas d'échec.
5. L'**API Cloud FastAPI** reçoit, authentifie et stocke dans PostgreSQL.
6. Le **Dashboard EdgePulse** visualise l'historique et compare les latences.

3.3. Namespace K3s

Namespace	Machine	Services
edge	Raspberry Pi 5	Microservices A, B, C

4. Simulateur IoT et Microservice A

4.1. Philosophie Edge Computing à la source

Implémentation d'un paradigme de traitement à la source sur l'**ESP32** : le nettoyage, le lissage et le formatage des données sont effectués directement sur le microcontrôleur avant toute transmission, réduisant la charge de traitement côté serveur.

Pour éviter les contraintes matérielles durant le sprint, un algorithme de génération de données pseudo-aléatoires contraintes (*Mocking*) a été développé.

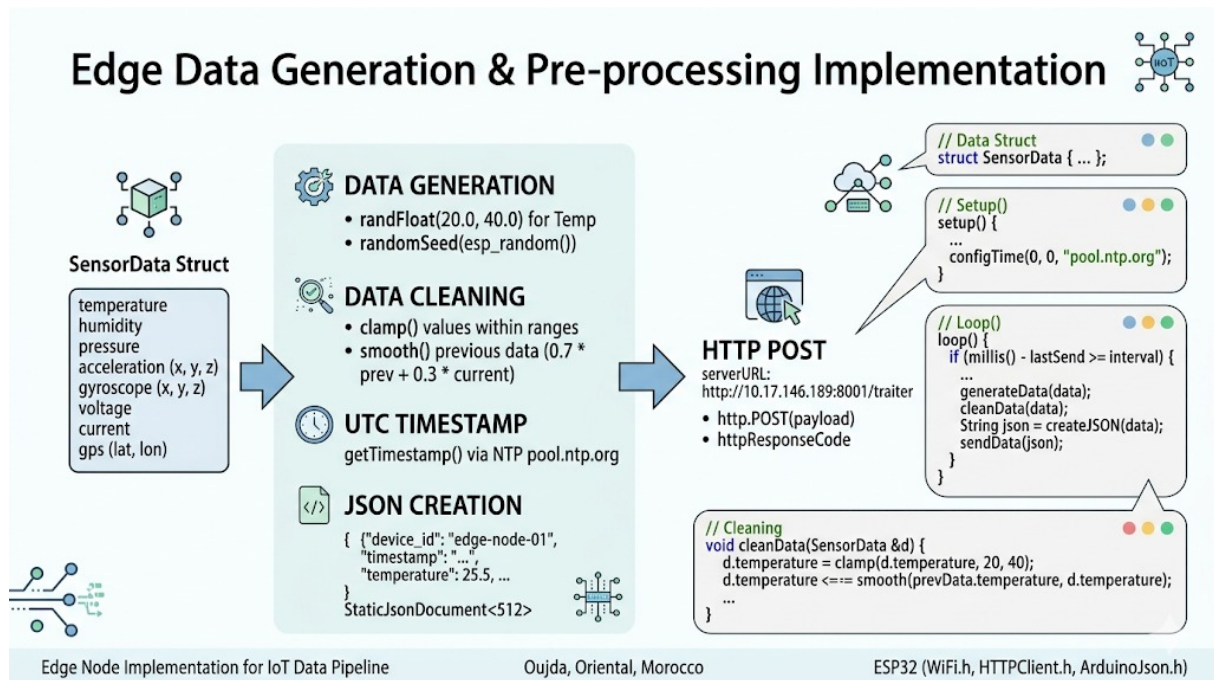


FIGURE 3 – Cycle global de création des données sur ESP32

4.2. Structure des données (Data Contract)

La structure C++ définie sur l'ESP32 constitue le contrat de données partagé avec tous les membres du groupe :

```

struct SensorData {
  float temperature;           // Température (°C)
  float humidity;              // Humidité (%)
  float pressure;              // Pression (hPa)
  float ax, ay, az;            // Accélération (m/s²)
  float gx, gy, gz;            // Gyroscope (rad/s)
  float voltage;               // Tension (V)
  float current;               // Courant (A)
  double lat;                  // GPS Latitude
  double lon;                  // GPS Longitude
};

```

4.3. Génération modélisée des données

La fonction `generateData()` utilise le générateur entropique matériel de l'ESP32 (`esp_random()`) pour produire des valeurs dans des plages physiques réalistes :

```
void generateData(SensorData &d) {
    d.temperature = randFloat(20.0, 40.0);
    d.humidity     = randFloat(30.0, 90.0);
    d.pressure     = randFloat(980.0, 1030.0);
    d.ax           = randFloat(-10.0, 10.0);
    // ... autres axes
    d.lat = 35.5765 + randFloat(-0.001, 0.001); // autour de Té
        touan
    d.lon = -5.3682 + randFloat(-0.001, 0.001);
}
```

4.4. Algorithmes de filtrage à la source

Deux algorithmes de qualité de données ont été implémentés sur l'ESP32 :

Écrêtage (Clamping)

Garantit de manière déterministe que les valeurs ne sortent jamais des seuils logiques, protégeant la base de données des valeurs aberrantes (*outliers*) :

```
float clamp(float v, float min, float max) {
    if (v < min) return min;
    if (v > max) return max;
    return v;
}
```

Lissage Exponentiel (EMA)

Pondère l'historique (70%) par rapport à la lecture brute (30%) pour éliminer les oscillations instables :

```
float smooth(float previous, float current) {
    return previous * 0.7f + current * 0.3f;
}
```

4.5. Sérialisation JSON et transmission

L'ESP32 sérialise les données via `ArduinoJson` avec allocation statique (`StaticJsonDocument<512>`) pour éviter la fragmentation mémoire :

```
String createJSON(SensorData &d) {
    StaticJsonDocument<512> doc;
    doc["device_id"] = "edge-node-01";
    doc["timestamp"] = getTimestamp(); // ISO 8601 UTC via NTP
    doc["temperature"] = d.temperature;
    // ...
}
```



```
JsonObject gps = doc.createNestedObject("gps");
gps["lat"] = d.lat;
gps["lon"] = d.lon;
String payload;
serializeJson(doc, payload);
return payload;
}
```

La transmission utilise une logique **non-bloquante** (`millis()`) garantissant un envoi toutes les 1000 ms sans bloquer le processeur, via HTTP POST avec header `Content-Type: application/json`.

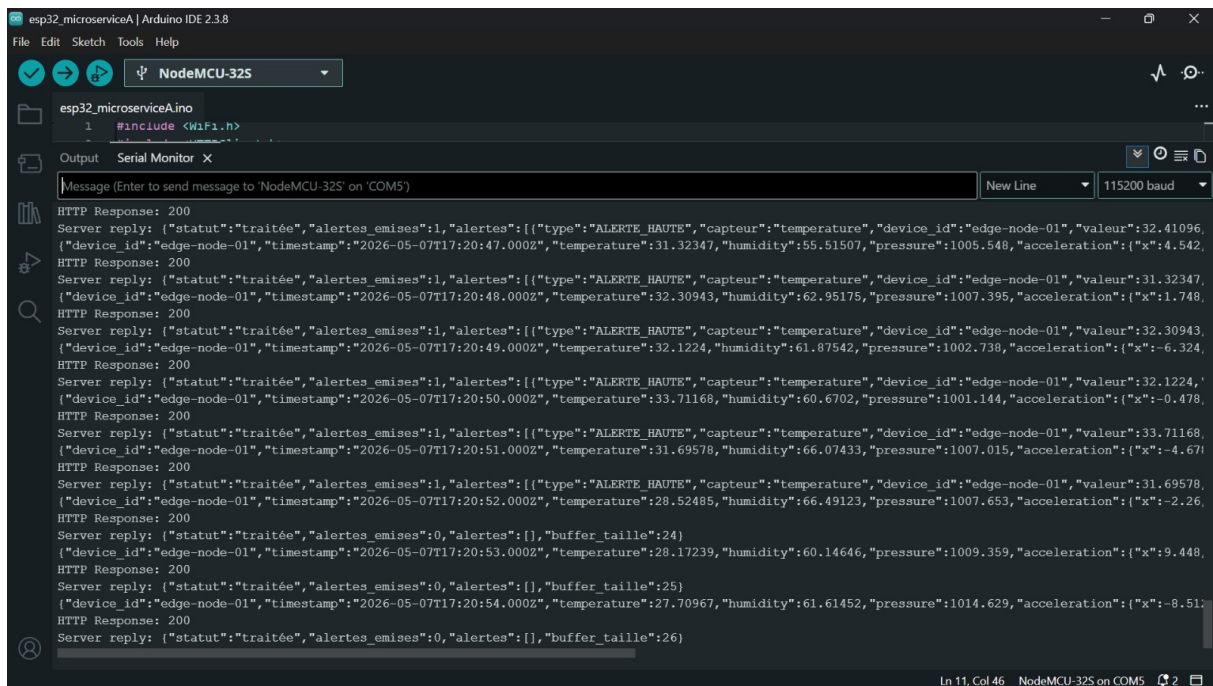


FIGURE 4 – Flux JSON dans le moniteur série

5. Infrastructure DevOps Edge (K3s)

5.1. Objectif

Le Membre 2 a transformé le Raspberry Pi 5 en nœud Kubernetes fonctionnel en utilisant K3s, une distribution légère adaptée aux machines à faibles ressources (ARM64).

5.2. Installation de K3s : Étapes détaillées

Étape 1 : Mise à jour du système

```
$ sudo apt update && sudo apt upgrade -y
```

Étape 2 : Configuration des cgroups

Étape cruciale : sans l'activation des *control groups*, K3s refuse de démarrer sur le Raspberry Pi. Le fichier de démarrage est modifié :

```
$ sudo nano /boot/firmware/cmdline.txt
# Ajouter à la fin de la ligne :
# cgroup_memory=1 cgroup_enable=memory
```

Étape 3 : Installation de K3s

```
$ curl -sfL https://get.k3s.io | sh -
```

L'installation automatique effectuée : téléchargement de K3s v1.35.4+k3s1 (ARM64), création du service systemd, et démarrage automatique.

Étape 4 : Vérification du nœud

```
$ sudo kubectl get nodes
NAME        STATUS    ROLES    AGE     VERSION
pi          Ready    control-plane  4m47s   v1.35.4+k3s1
```

```
pi@Pi:~ $ sudo kubectl create namespace edge
namespace/edge created
pi@Pi:~ $ sudo kubectl get namespaces
NAME                STATUS    AGE
default             Active    18m
edge                Active    20s
kube-node-lease     Active    18m
kube-public         Active    18m
kube-system         Active    18m
```

FIGURE 5 – Nodes

Étape 5 : Création du namespace edge

```
$ sudo kubectl create namespace edge
namespace/edge created

$ sudo kubectl get namespaces
NAME                STATUS    AGE
default             Active    18m
edge                Active    20s
kube-system         Active    18m
```

```
pi@Pi:~ $ sudo kubectl create namespace edge
namespace/edge created
pi@Pi:~ $ sudo kubectl get namespaces
NAME                STATUS    AGE
default             Active    18m
edge                Active    20s
kube-node-lease     Active    18m
kube-public         Active    18m
kube-system         Active    18m
```

FIGURE 6 – Namespaces

5.3. Installation de Docker

Docker est installé pour permettre le build des images sur l'architecture ARM64 avant déploiement dans K3s :

```
$ curl -fsSL https://get.docker.com | sh
$ sudo usermod -aG docker $USER
$ docker --version
Docker version 29.4.3, build 055a478
$ docker run hello-world
```

```
pi@Pi:~$ sudo usermod -aG docker $USER
pi@Pi:~$ newgrp docker
pi@Pi:~$ docker --version
Docker version 29.4.3, build 055a478
pi@Pi:~$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:f9078146db2e05e794366b1bfe584a14ea6317f44027d10ef7dad65279026885
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
pi@Pi:~$
```

FIGURE 7 – Docker installé

5.4. Déploiement des microservices

Pour chaque microservice (A, B, C), la procédure est identique :

```
$ docker build -t microservice-b:latest .      # Build ARM64
$ sudo kubectl apply -f deployment.yaml        # Déploiement K3s
$ sudo kubectl get pods -n edge                # Vérification
```

Les images Docker sont construites directement sur le Pi (ARM64) pour assurer la compatibilité avec l'architecture du processeur. Une image buildée sur x86 ne fonctionnerait pas sur le Pi sans émulation.

5.5. Distinction Docker vs K3s

	Docker	K3s
Rôle	Tests locaux	Production
Utilisation	Builder une image	Déployer en cluster
Runtime	Docker Engine	containerd
Commande	docker run	kubectl apply

6. Microservice B : Traitement Edge

6.1. Rôle dans l'architecture

Le Microservice B est le cœur du traitement Edge. Il reçoit les données brutes du Microservice A, détecte les anomalies localement sans Internet (<10 ms), et agrège les données sur 5 minutes pour le Microservice C.

Composant	Rôle	Protocole
POST /traiter	Reçoit les données brutes du MS-A	HTTP POST (JSON)
Alert Detection	Compare aux seuils configurés	Local (RAM)
Memory Buffer	Stocke 300 lectures (5 min)	In-memory (deque)
GET /agregat	Retourne les moyennes au MS-C	HTTP GET (JSON)
GET /alertes	Historique des alertes	HTTP GET (JSON)
GET /health	Liveness probe K3s	HTTP GET (JSON)

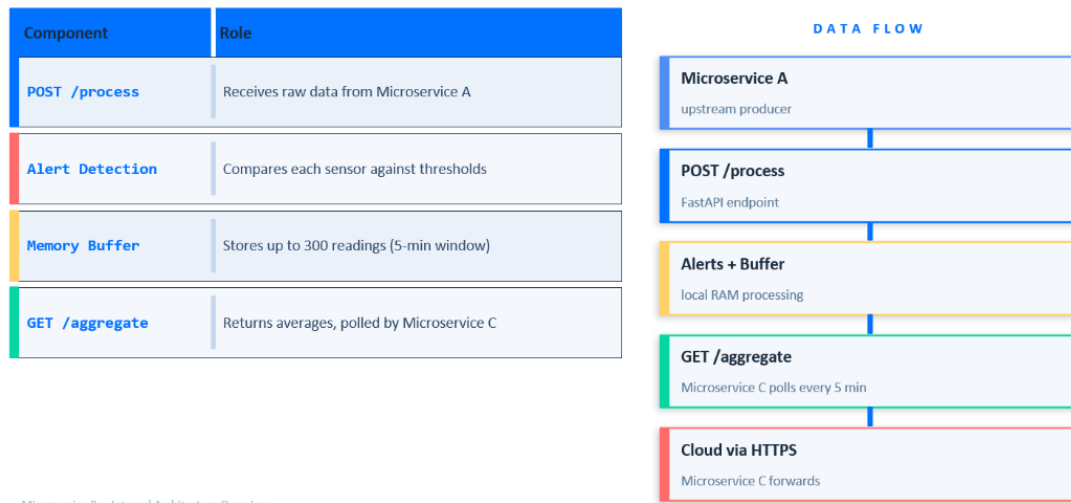


FIGURE 8 – Vue d'ensemble du microservice

6.2. Détection d'alertes : `detecter_alertes()`

La fonction `detecter_alertes()` analyse les 5 capteurs scalaires à chaque réception de données. Les seuils configurés sont :

Capteur	Minimum	Maximum	Unité
temperature	10.0	30.0	°C
humidity	20.0	85.0	%
pressure	950.0	1050.0	hPa
voltage	2.5	5.5	V
current	0.0	3.0	A

```

SEUILS = {
    "temperature": {"min": 10.0, "max": 30.0},
    "humidity": {"min": 20.0, "max": 85.0},
    "pressure": {"min": 950.0, "max": 1050.0},
    "voltage": {"min": 2.5, "max": 5.5},
    "current": {"min": 0.0, "max": 3.0},
}

def detecter_alertes(donnee: DonneeesBrutes) -> list:
    result = []
    champs = {
        "temperature": donnee.temperature,
        "humidity": donnee.humidity,
        "pressure": donnee.pressure,
        "voltage": donnee.voltage,
        "current": donnee.current,
    }
    for champ, valeur in champs.items():
        seuil = SEUILS[champ]
        if valeur > seuil["max"]:
            result.append({
                "type": "ALERTE_HAUTE",
                "capteur": champ,
                "valeur": valeur,
                "message": f"{champ} trop élevé : {valeur} (max {seuil['max']})",
            })
    return result

```

6.3. Agrégation : calculer_agregat()

Le mécanisme de **snapshot avant vidage** est crucial dans un contexte concurrent :

```

def calculer_agregat() -> Optional[dict]:
    if not buffer:
        return None
    snap = list(buffer)      # Snapshot instantané
    buffer.clear()           # Vide pour les nouvelles mesures
    agregat = {
        "device_id": snap[-1]["device_id"],
        "timestamp": datetime.now(timezone.utc).isoformat(),
        "nb_mesures": len(snap),
        "temperature": round(statistics.mean(
            d["temperature"] for d in snap), 2),
        # ... autres capteurs
    }
    return agregat

```

Le snapshot garantit qu'aucune donnée n'est perdue lors du calcul des agrégats, même si de nouvelles mesures arrivent pendant le traitement.

6.4. Endpoint POST /traiter

```
@app.post("/traiter")
def recevoir(donnee: DonneesBrutes):
    # 1. Détection alertes (chemin rapide, local, < 10 ms)
    nouvelles_alertes = detecter_alertes(donnee)
    for a in nouvelles_alertes:
        alertes.append(a)
        print(f"[ALERTE] {a['message']}")
    # 2. Mise en buffer pour agrégation
    buffer.append({
        "temperature": donnee.temperature,
        "humidity":     donnee.humidity,
        # ...
    })
    return {
        "statut":          "traitée",
        "alertes_emises":  len(nouvelles_alertes),
        "buffer_taille":   len(buffer),
    }
```

6.5. Déploiement et liveness probe

La liveness probe K3s interroge /health toutes les 15 secondes. Si l'API ne répond plus, K3s redémarre automatiquement le conteneur :

```
livenessProbe:
  httpGet:
    path: /health
    port: 8001
  initialDelaySeconds: 10
  periodSeconds: 15
```

7. API Cloud et VM Azure

7.1. Infrastructure Azure

Plutôt que de simuler le Cloud dans un namespace K3s sur le Pi, nous avons déployé un vrai serveur distant sur Microsoft Azure. Cela permet une mesure authentique de la latence : le paquet de données transite réellement entre Tétouan et un datacenter en Afrique du Sud à l'autre bout du continent.

Caractéristique	Valeur
Fournisseur	Microsoft Azure
Taille	Standard_B2ats_v2 (2 vCPU)
OS	Ubuntu 24.04 LTS
Région	South Africa North
IP publique	20.164.76.70
Domaine	cloudserver-g09.southafricanorth.cloudapp.azure.com

Machine virtuelle

Nom de l'ordinateur CloudServer

Système d'exploitation Linux (ubuntu 24.04)

Génération de machine virtuelle V2

Architecture de machine virtuelle x64

État de l'agent Ready

Version de l'agent 2.15.1.3

Mise en veille prolongée Désactivé

Groupe hôte -

Hôte -

Groupe de placement de proximité -

État de colocation N/A

Groupe de réservations de capacité -

Mise en réseau

Adresse IP publique 20.164.76.70 (Interface cloudserver840_z1)
réseau

1 adresses IP publiques associées

Adresse IP publique (IPv6) -

Adresse IP privée 10.0.0.4

Adresse IP privée (IPv6) -

Réseau/sous-réseau virtuel CloudServer-vnet/default

Nom DNS cloudserver-g09.southafricanorth.cloudapp.azure.com

Taille

Taille Standard B2ats v2

Processeurs virtuels 2

RAM 1 Gio

FIGURE 9 – VM Azure

7.2. Stack déployée

Service	Port interne	Exposition externe
FastAPI	8000	Non : via Nginx uniquement
PostgreSQL	5432	Non : interne uniquement
Nginx	80, 443	Oui : Internet (NSG Azure)

```
$ docker compose ps
```

NAME	IMAGE	PORTS
cloud-api-api-1	cloud-api-api	127.0.0.1:8000->8000/tcp
cloud-api-postgres-1	postgres:16	127.0.0.1:5432->5432/tcp


```
cloudserver@CloudServer:~/cloud-api$ docker compose ps
WARN[0000] /home/cloudserver/cloud-api/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PORTS
cloud-api-api-1      cloud-api-api        "uvicorn main:app --"  api        15 hours ago   Up 15 hours   127.0.0.1:8000->8000/tcp
cloud-api-postgres-1 postgres:16-alpine    "docker-entrypoint.s..." postgres   15 hours ago   Up 15 hours   127.0.0.1:5432->5432/tcp
cloudserver@CloudServer:~/cloud-api$
```

FIGURE 10 – Docker

7.3. API FastAPI : Endpoints

POST /data : Réception des données

Reçoit le payload JSON du Microservice C, valide via Pydantic, écrit en PostgreSQL.
Header requis : x-api-key: edge-secret-key-2026

```
{
  "device_id":      "edge-node-01",
  "timestamp":      "2026-05-05T14:32:01.123Z",
  "temperature":    27.4,  "humidity": 61.2,  "pressure": 1013.5,
  "acceleration":   {"x": 0.12, "y": -0.03, "z": 9.81},
  "gyroscope":      {"x": 0.001, "y": 0.002, "z": -0.001},
  "voltage":        3.28,  "current": 0.45,
  "gps":            {"lat": 35.5765, "lon": -5.3682}
}
```

GET /data : Lecture de l'historique

Expose les données pour le Dashboard. Paramètres optionnels : limit, device_id, since.

GET /health : Liveness probe

Retourne {"status": "ok"} sans authentification.

Cloud API — Projet Edge Computing 1.0.0 OAS 3.1

/openapi.json

Reçoit, stocke et expose les données des capteurs IoT depuis le nœud Edge.

default		^
POST	/data Receive Data	▼
GET	/data Get Data	▼
GET	/health Health	▼

FIGURE 11 – Vue du doc swagger

7.4. Schéma de données PostgreSQL

Les 9 capteurs sont stockés dans la table **sensor_data**. Les valeurs scalaires ont des colonnes **float** dédiées. Les objets imbriqués (accélération, gyroscope, GPS) sont stockés en JSONB PostgreSQL pour permettre l'indexation.

```

class SensorRecord(Base):
    __tablename__ = "sensor_data"
    id = Column(Integer, primary_key=True)
    device_id = Column(String, index=True)
    timestamp = Column(DateTime, index=True)
    temperature = Column(Float)
    humidity = Column(Float)
    pressure = Column(Float)
    voltage = Column(Float)
    current = Column(Float)
    acceleration = Column(JSONB) # {x, y, z}
    gyroscope = Column(JSONB) # {x, y, z}
    gps = Column(JSONB) # {lat, lon}

```

8. Sécurisation des communications

8.1. Architecture de sécurité

La sécurisation repose sur trois niveaux indépendants et complémentaires.

Niveau 1 : Chiffrement TLS (HTTPS)

Le certificat SSL est émis par **Let's Encrypt** via Certbot, directement sur la VM Azure. Il est reconnu nativement par tous les clients HTTP sans configuration particulière. Toutes les données transitent chiffrées sur Internet : sans HTTPS, les mesures capteurs et la clé API transitent en clair.

```

$ sudo certbot --nginx -d cloudserver-g09.southafricanorth.
  cloudapp.azure.com
$ sudo certbot certificates
Found the following certs:
  Certificate Name: cloudserver-g09.southafricanorth.cloudapp.
    azure.com
  Expiry Date: ...
  Certificate Path: /etc/letsencrypt/live/.../fullchain.pem

```

Niveau 2 : Authentification par clé API

Chaque requête vers POST /data et GET /data doit inclure :

```
x-api-key: edge-secret-key-2026
```

Sans ce header, l'API retourne immédiatement 401 Unauthorized sans toucher à la base de données.

Niveau 3 : Isolation réseau (NSG Azure)

Le port 8000 de FastAPI est fermé dans le pare-feu Azure (NSG). Tout le trafic passe obligatoirement par Nginx sur le port 443 : le chiffrement TLS est donc toujours appliqué,

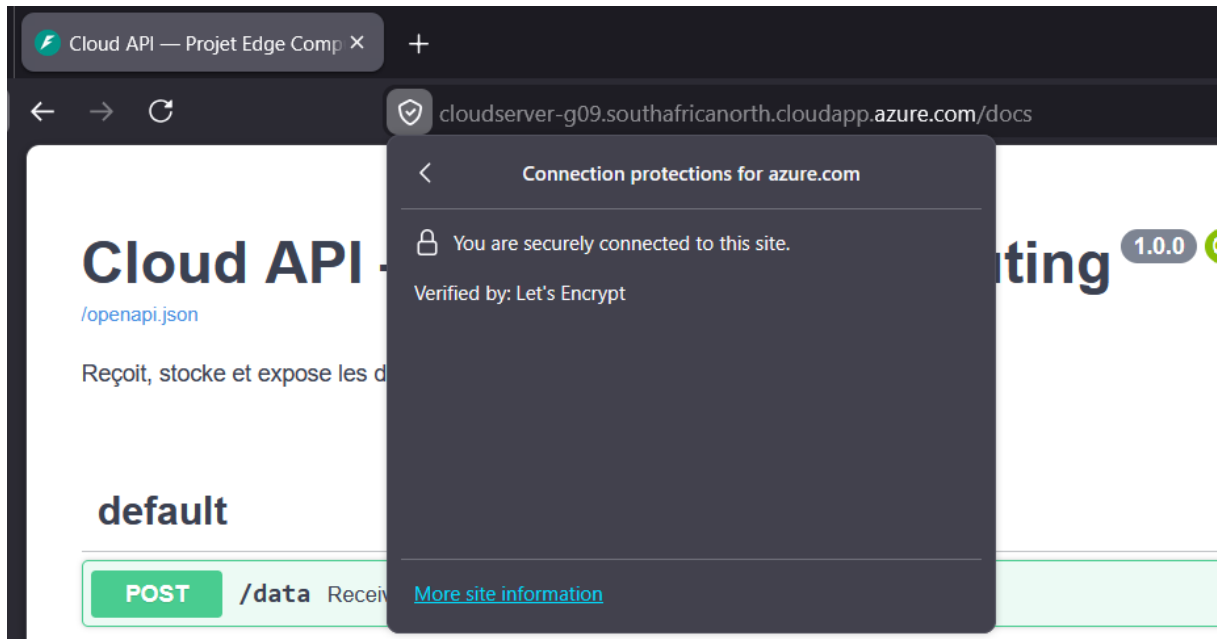


FIGURE 12 – Communications chiffrées

sans exception possible.

```
Architecture du flux entrant :  
Internet (HTTPS 443)  
  --> Nginx (certificat Let's Encrypt)  
    --> FastAPI (localhost:8000)  
      --> PostgreSQL (localhost:5432)
```

8.2. Configuration Nginx

```
server {  
    listen 443 ssl;  
    server_name cloudserver-g09.southafricanorth.cloudapp.azure  
        .com;  
    location / {  
        proxy_pass http://localhost:8000;  
        proxy_set_header Host $host;  
        proxy_set_header X-Forwarded-Proto $scheme;  
    }  
}
```

9. Microservice C et Dashboard EdgePulse

9.1. Microservice C : Synchronisation Edge-Cloud

Le Microservice C est le pont sécurisé entre le Raspberry Pi et Azure. Il ne traite pas les données : il les transporte fidèlement toutes les 5 minutes.

Cycle d'exécution

1. Interroger `GET /agregat` sur le Microservice B : récupère la moyenne des 5 dernières minutes et vide le buffer.
2. Envoyer l'agrégat à l'API Cloud Azure via `POST /data` en HTTPS.
3. Attendre 5 minutes et recommencer.

```
API_URL = "https://cloudserver-g09.southafricanorth.cloudapp.azure.com"
API_KEY = "edge-secret-key-2026"

def send_to_cloud(payload: dict):
    response = requests.post(
        f"{API_URL}/data",
        json=payload,
        headers={"x-api-key": API_KEY},
        timeout=10
    )
    response.raise_for_status()
    return response.json()
```

Backoff exponentiel : Tolérance aux pannes

En cas de coupure réseau, le Microservice C réessaie automatiquement sans perdre les données :

Tentative	Délai d'attente
Échec 1	2 secondes
Échec 2	4 secondes
Échec 3	8 secondes
Échec 4	16 secondes
Échec 5	32 secondes puis abandon

9.2. Dashboard EdgePulse

Le Dashboard EdgePulse est développé avec **Streamlit** (Python pur, sans HTML/JS) et **Plotly** pour les graphiques interactifs.

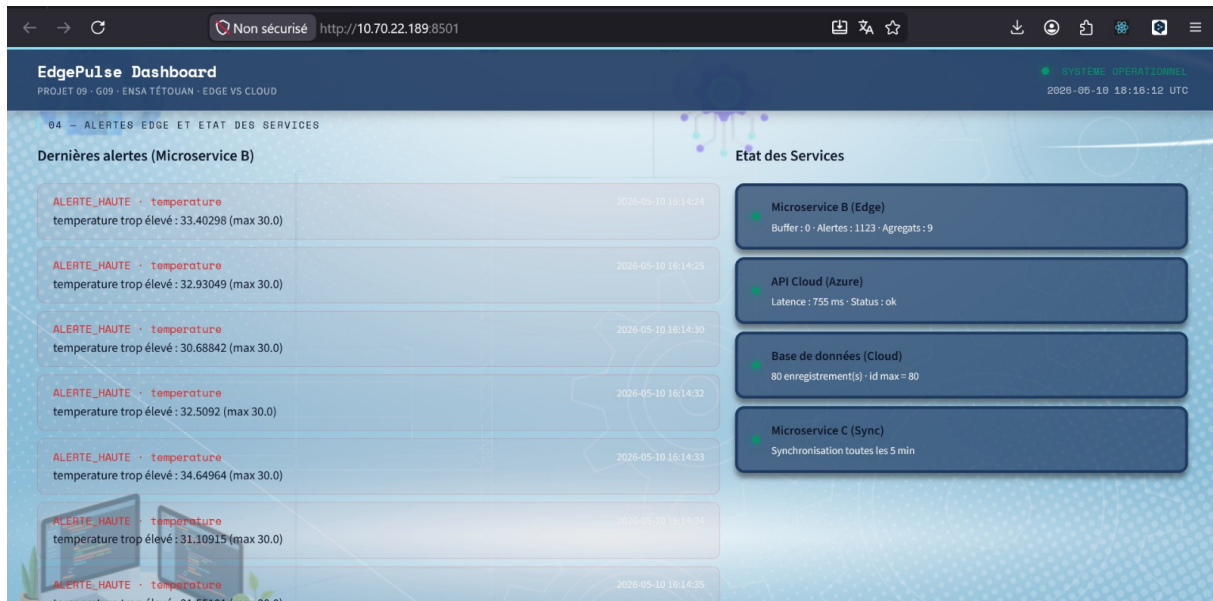


FIGURE 13 – Dashboard

Sources de données

Fonction	Source
<code>fetch_cloud_data()</code>	GET <code>https://cloudserver.../data</code>
<code>fetch_b_health()</code>	GET <code>http://<Pi>:8001/health</code>
<code>fetch_alerts()</code>	GET <code>http://<Pi>:8001/alertes</code>
<code>measure_edge_latency()</code>	5 pings vers Microservice B, moyenne
<code>measure_cloud_latency()</code>	5 pings vers API Azure, moyenne

Sections du dashboard

- **Section 01 :** Cartes de latence Edge vs Cloud, bar chart logarithmique, historique de session
- **Section 02 :** 6 KPI cards (température, humidité, pression, tension, courant, GPS) avec statistiques min/max/moyenne
- **Section 03 :** 6 graphiques Plotly sur fenêtre glissante de 2 heures
- **Section 04 :** Historique des alertes + état des 4 services
- **Section 05 :** Tableau des 15 dernières mesures

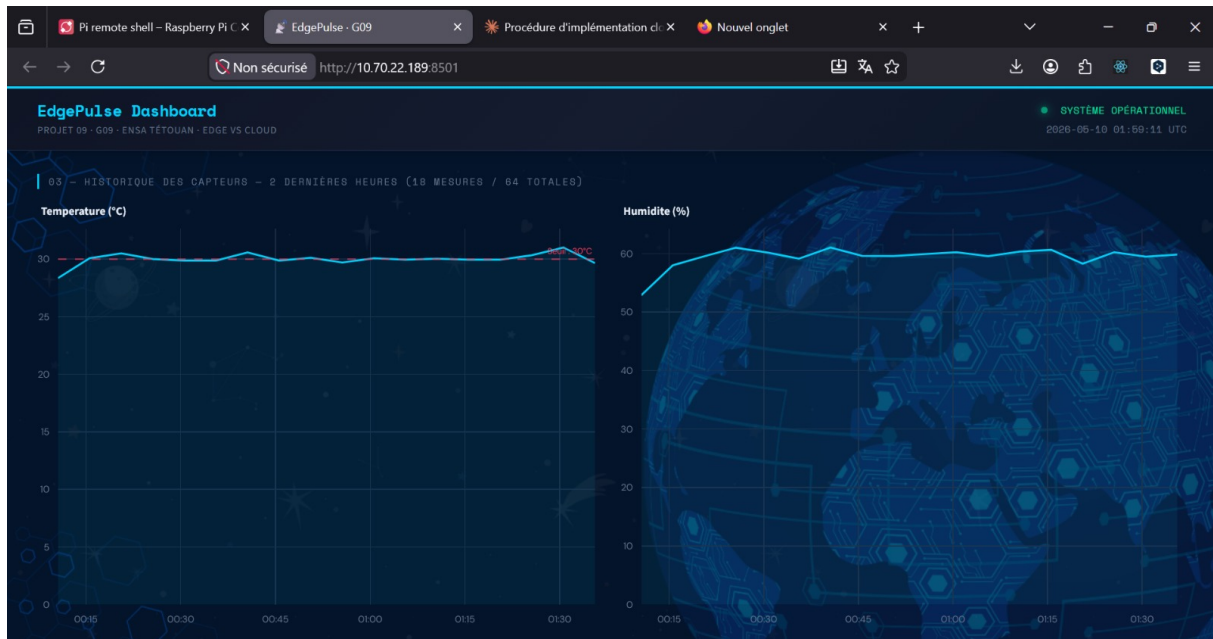


FIGURE 14 – Vue de la section graphiques

Mesure de la latence

```
def measure_edge_latency(n=5):
    latencies = []
    for _ in range(n):
        t0 = time.time()
        requests.get("http://<IP-Pi>:8001/health")
        latencies.append((time.time() - t0) * 1000)
        time.sleep(0.05)
    return sum(latencies) / len(latencies)
```

Fonctionnalités techniques

- **Cache** @st.cache_data(ttl=20) : évite les requêtes HTTP à chaque interaction utilisateur : les données sont mises en cache 20 secondes.
- **Auto-refresh toutes les 30 secondes** : st.rerun() recharge la page complète automatiquement.
- **Mode sombre/clair** : dictionnaire de thème appliqué dynamiquement à tous les composants CSS et graphiques Plotly.
- **Historique de session** : st.session_state conserve l'historique des latences entre les rechargements (60 points max, 30 min).

10. Résultats et comparaison des performances

10.1. Latences mesurées

Type de traitement	Latence mesurée	Localisation
Traitement Edge (Microservice B)	<10 ms	Raspberry Pi (local)
Communication Cloud (Azure)	~670 ms	Pi → Afrique du Sud

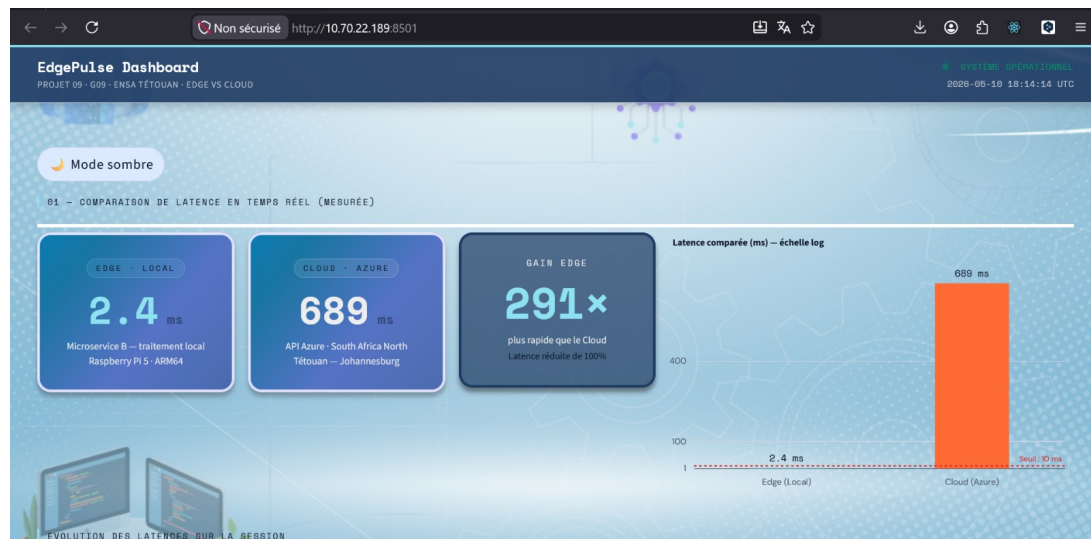


FIGURE 15 – Comparaison Edge et Cloud

10.2. Analyse

La latence Cloud de ~670 ms se décompose ainsi :

- Transit réseau Tétouan → Johannesburg : ~300 ms
- Traitement serveur : ~50 ms
- Transit retour : ~300 ms

Le **gain apporté par l'Edge** est calculé dynamiquement par le dashboard :

```
ratio      = cloud_lat / edge_lat    # 670 / 3 = ~223x
reduction  = (cloud_lat - edge_lat) / cloud_lat * 100 # ~99.5%
```

10.3. Réduction du trafic réseau

L'agrégation sur 5 minutes dans le Microservice B réduit le trafic réseau d'un facteur 300 : 300 mesures brutes par seconde sont condensées en 1 seul agrégat envoyé toutes les 5 minutes vers Azure.

11. Conclusion

Ce projet a permis de déployer une architecture hybride Edge-Cloud complète et fonctionnelle, démontrant concrètement les bénéfices du Edge Computing :

- **Réduction de latence** : de >670 ms (Cloud) à <10 ms (Edge), soit un gain d'un facteur ~ 220 .
- **Réduction du trafic** : d'un facteur 300 grâce à l'agrégation locale.
- **Sécurité bout en bout** : HTTPS avec certificat Let's Encrypt valide, authentification par clé API, isolation réseau via NSG Azure.
- **Haute disponibilité** : K3s assure le redémarrage automatique de tous les microservices via les liveness probes.
- **Architecture réelle** : contrairement à une simulation, la communication Edge-Cloud passe par Internet réel (Tétouan $\rightarrow$ Afrique du Sud).

L'approche Scrum a permis de livrer un système complet et intégré en une semaine de sprint, avec une collaboration efficace entre les 5 membres malgré les dépendances séquentielles entre les composants.

Dépôt GitHub

Le code source complet du projet est disponible sur le dépôt GitHub du groupe :

<https://github.com/KunakaDK/Distributed-Edge-Nodes>